



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Camila Raquel Marcos Medinilla **Carné:** 1120525

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable



Summary - Overview x milamarc707/serviceconnector- x +

github.com/milamarc707/serviceconnector-webapp-storageblob-dotnet

milamarc707 / serviceconnector-webapp-storageblob-dotnet

Code Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

serviceconnector-webapp-storageblob-dotnet Public

forked from Azure-Samples/serviceconnector-webapp-storageblob-dotnet

main 1 Branch 0 Tags

Go to file Add file Code

This branch is up to date with Azure-Samples/serviceconnector-webapp-storageblob-dotnet:main

Contribute Sync fork

yungezz upgrade to dotnet6 (Azure-Samples#4) d6ec11d · 3 years ago 14 Commits

File	Commit	Time
.github	.github/PULL_REQUEST_TEMPLATE.md committed	5 years ago
Pages	initial version	5 years ago
Properties	initial version	5 years ago
wwwroot	initial version	5 years ago
.qitignore	initial version	5 years ago

About

Sample of using Service Connector to connect Azure WebApp to Azure Storage Blob

Readme License Code of conduct Contributing Activity 0 stars 0 watching 0 forks

Releases

07:25 7/10/2026

2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

File Edit Selection View Go Run ...

semana-14

Update

Explorer

- semana-14
 - serviceconnector-webapp-storageblob-dot...
 - .gitkeep

Start

- New File...
- Open File...
- Open Folder...
- Clone Git Repository...
- Connect to...

Walkthroughs

- Get started with VS Code
- Learn the Fundamentals

Problems Output Debug Console Terminal Ports

PS C:\Users\camil\Desktop\url-mia-2026\semana-1\semana-14> git clone https://github.com/milamarc707/serviceconnector-webapp-storageblob-dotnet.git

Cloning into 'serviceconnector-webapp-storageblob-dotnet'...

remote: Enumerating objects: 115, done.

remote: Counting objects: 100% (115/115), done.

remote: Compressing objects: 100% (93/93), done.

remote: Total 115 (delta 34), reused 77 (delta 16), pack-reused 0 (from 0)

Receiving objects: 100% (115/115), 697.62 KiB | 2.59 MiB/s, done.

Resolving deltas: 100% (34/34), done.

PS C:\Users\camil\Desktop\url-mia-2026\semana-1\semana-14> dir

Directorio: C:\Users\camil\Desktop\url-mia-2026\semana-1\semana-14

Mode	LastWriteTime	Length	Name
d----	7/10/2026 07:31		serviceconnector-webapp-storageblob-dotnet
-a----	20/07/2026 08:03		.gitkeep

07:32 7/10/2026



3. Crear DotNet Env

- En terminal, instalar: **dotnet add package DotNetEnv**
- Crear .env file en raíz de proyecto, valores ->
AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default
- Crear .env.production con los mismos valores en el mismo lugar
- Añadir cambio brindado en program.cs
- dotnet clean -> dotnet restore -> dotnet build -> dotnet run

```
1 using DotNetEnv;
2 using Microsoft.AspNetCore.Hosting;
3 using Microsoft.Extensions.Hosting;
4
5 namespace WebStorageSample
6 {
7     0 references
8     public class Program
9     {
10         0 references
11         public static void Main(string[] args)
12         {
13             Env.Load(".env");
14             CreateHostBuilder(args).Build().Run();
15         }
16
17         1 reference
18         public static IHostBuilder CreateHostBuilder(string[] args) =>
19         {
20             Host.CreateDefaultBuilder(args)
21                 .ConfigureWebHostDefaults(webBuilder =>
22                 {
23                     webBuilder.UseStartup<Startup>();
24                 })
25                 .Build();
26         }
27     }
28 }
```



4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → tu Storage Account → Data storage → Containers
- Crear contenedor "default"
- Activa identidad administrada en App Service: Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)
- Configurar la conexión entre App Service y Blob Storage: App Service → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create.



The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Azure logo, a search bar, and the user profile. The main content area displays the 'storagecontainerconnectorcrrmm' service connector configuration. A left sidebar contains a navigation menu with options like 'Introducción', 'Registro de actividad', 'Control de acceso (IAM)', 'Etiquetas', 'Diagnosticar y solucionar problemas', 'Microsoft Defender for Cloud', 'Eventos (versión preliminar)', 'IA', 'Visualizador de recursos', 'Implementación', 'Configuración', and 'Rendimiento'. The main area shows a table with columns: 'Tipo de servicio', 'Nombre del recurso', 'Nombre de la conexión', and 'Estado de la conexión'. The table contains one entry: 'Blob Storage' with resource name 'crrmmstoragesemana12' and connection name 'storageblob_91c6d', with a status of 'Sin validar'.

Tipo de servicio	Nombre del recurso	Nombre de la conexión	Estado de la conexión
Blob Storage	crrmmstoragesemana12	storageblob_91c6d	Sin validar

e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service

The screenshot shows a Visual Studio Code editor with a project named 'semana-14'. The Explorer sidebar on the left shows the file structure, including a '.env' file. The main editor area displays the content of the '.env' file, which contains the following lines:

```
1 AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://crrmmstoragesemana12.blob.core.windows.net/  
2 AZURE_STORAGE_CONTAINER_NAME=default  
3
```

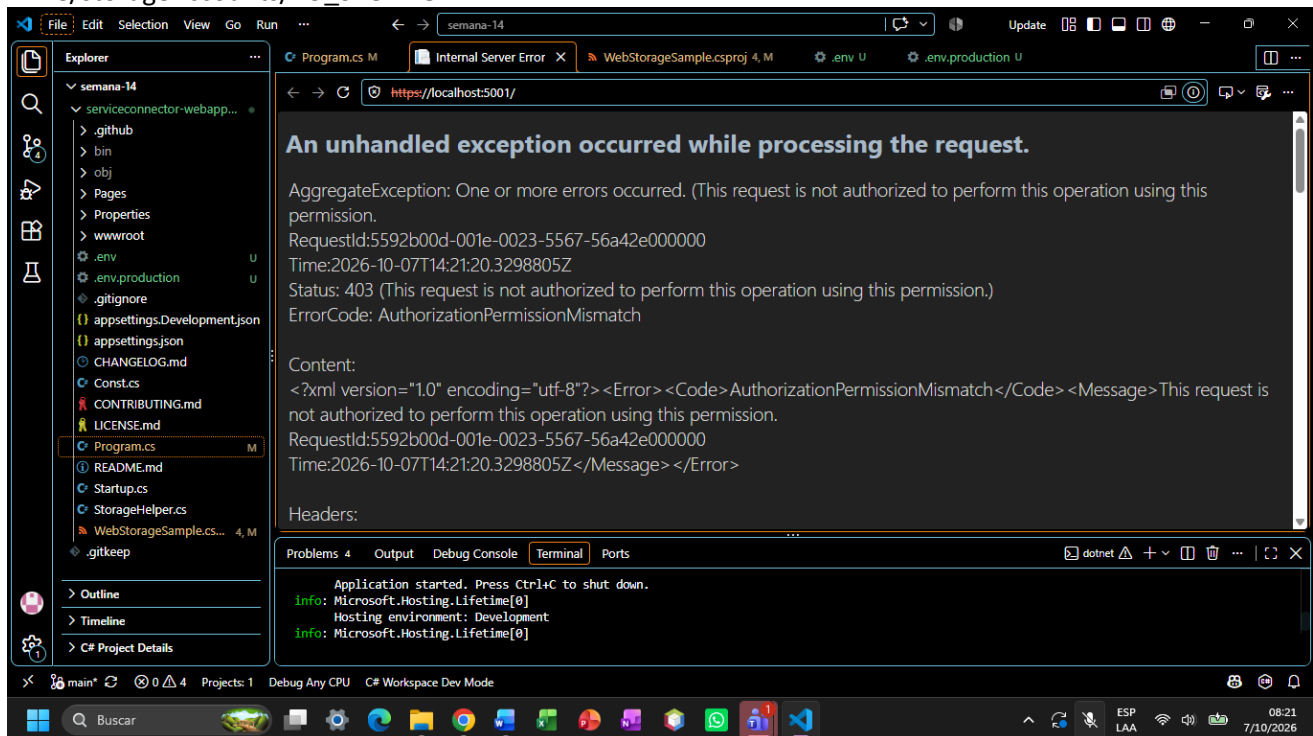
Below the editor, the 'Terminal' panel shows the output of a PowerShell command. The output includes information about the application's lifetime, listening ports (https://localhost:5001 and http://localhost:5000), and the application's startup and shutdown messages. The application is running on the path 'C:\Users\camil\Desktop\url-mia-2026\semana-1\semana-14\serviceconnector-webapp-storageblob-dotnet'.



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`
- Verifique que pueda obtener un token para Azure Storage: `az account get-access-token --resource https://storage.azure.com/`
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> `dotnet run`
- Si hay error en acceso, ejecutar en terminal vs code: `az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"`



- Obtener info:** `az account show --query "{subscriptionId:id, user:user.name}" -o table`
`az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table`
 g) Cerrar process `ctr +c` y logear de nuevo: `az logout` y `az login`, Por ultimo `dotnet run`

Nota: no olvidar subir cambios a repo github fork

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:
- main

pool:
vmImage: 'windows-latest'

variables:
buildConfiguration: 'Release'



```
# Completar posteriormente
azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
appName: 'NOMBRE-DE-TU-APP-SERVICE'
```

steps:

1. Instalar .NET 9

```
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'
```

2. Restaurar dependencias

```
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'
```

3. Configurar ambiente de producción

```
- powershell: |
  Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'
```

4. Compilar

```
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'
```

5. Publicar

```
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
  zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
```



8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps con misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**
- Probar seleccionando "run"
- Conectar Azure DevOps con Azure: **Project Settings -> Service connections -> New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save**

8. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services -> tu aplicación -> Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name

appName: Tu_app_name

- Al hacer commit, verificar el pipeline se ejecuta automáticamente

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

- acceso de la aplicación al Blob: Azure Portal -> App Service -> Settings -> Environment variables -> App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
- Verificar app: Azure Portal -> App Service -> OverView -> Default Domain -> click en URL publica
- En la app publicada realizar cambios en archivos
- Verificar que los cambios se representen en storage account: Azure Portal -> Storage Account -> Storage Browser -> Blob containers -> Seleccionar su container -> Verificar cambios

10. Preguntas de analisis

- ¿Para qué sirve el archivo .env en la aplicación?
- ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
- ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
- ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
- ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
- ¿Qué ocurre después de hacer push a main?
- Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.